Experiment 5

Student Name: Ayush Yadav **UID: 25IMC13004**

Branch: MCA (AI & ML) **Section/Group: 25MAM_KAR-1_A**

Semester: II **Date of Performance: 24/02/26**

Subject Name: Technical Training **Subject Code: 25CAP-652**

Experiment Aim: To gain hands-on experience in creating and using cursors for row-by-row processing in a database, enabling sequential access and manipulation of query results for complex business logic.

Tools used: Oracle / MS SSMS / PostgreSQL

Objectives:

- **Sequential Data Access:** To understand how to fetch rows one by one from a result set using cursor mechanisms.
- **Row-Level Manipulation:** To perform specific operations or calculations on individual records that require conditional procedural logic.
- **Resource Management:** To learn the lifecycle of a cursor: Declaring, Opening, Fetching, and importantly, Closing and Deallocating to manage system memory.
- **Exception Handling:** To handle cursor-related errors and performance considerations during large-scale data iteration.

Experiment Steps:

```
CREATE TABLE Employee (
  emp_id NUMBER,
  emp_name VARCHAR2(50),
  salary NUMBER
);
INSERT INTO Employee VALUES (1,'Rahul',20000);
INSERT INTO Employee VALUES (2,'Amit',0);
INSERT INTO Employee VALUES (3,'Neha',25000);
INSERT INTO Employee VALUES (4,'Anay',20000);
```

Step 1: Implementing a Simple Forward-Only Cursor

Creating a cursor to loop through an Employee table and print individual records.

```
DECLARE
  CURSOR emp_cursor IS
    SELECT emp_id, emp_name, salary FROM Employee;
  v_emp_id  Employee.emp_id%TYPE;
  v_emp_name Employee.emp_name%TYPE;
  v_salary  Employee.salary%TYPE;
BEGIN
```

```
    OPEN emp_cursor;
    FETCH emp_cursor INTO v_emp_id, v_emp_name, v_salary;
    WHILE emp_cursor%FOUND
    LOOP
      DBMS_OUTPUT.PUT_LINE(
        'ID: ' || v_emp_id ||
        ' Name: ' || v_emp_name ||
        ' Salary: ' || v_salary
      );
      FETCH emp_cursor INTO v_emp_id, v_emp_name, v_salary;
    END LOOP;
    CLOSE emp_cursor;
  END;
```

```
ID: 1 Name: Rahul Salary: 20000
ID: 2 Name: Amit Salary: 0
ID: 3 Name: Neha Salary: 25000
ID: 4 Name: Anay Salary: 20000
ID: 4 Name: Anay Salary: 20000
```

Step 2: Complex Row-by-Row Manipulation

Using a cursor to update salaries based on a dynamic "Experience-to-Performance" ratio logic.

```
  DECLARE
    CURSOR emp_cursor IS
      SELECT emp_id, salary FROM Employee FOR UPDATE;
    v_emp_id Employee.emp_id%TYPE;
    v_salary Employee.salary%TYPE;
  BEGIN
    OPEN emp_cursor;
    LOOP
      FETCH emp_cursor INTO v_emp_id, v_salary;
      EXIT WHEN emp_cursor%NOTFOUND;
      IF v_salary < 20000 THEN
        UPDATE Employee
        SET salary = v_salary * 1.20
        WHERE CURRENT OF emp_cursor;
      ELSIF v_salary BETWEEN 20000 AND 30000 THEN
        UPDATE Employee
        SET salary = v_salary * 1.10
        WHERE CURRENT OF emp_cursor;
      ELSE
        UPDATE Employee
```

```
        SET salary = v_salary * 1.05
        WHERE CURRENT OF emp_cursor;
      END IF;
    END LOOP;
    CLOSE emp_cursor;
    DBMS_OUTPUT.PUT_LINE('Salary Updated Successfully.');
END;
```

```
Salary Updated Successfully.
```

```
ID: 1 Name: Rahul Salary: 22000
ID: 2 Name: Amit Salary: 0
ID: 3 Name: Neha Salary: 27500
ID: 4 Name: Anay Salary: 22000
```

Step 3: Exception and Status Handling

Ensuring the cursor handles empty result sets or termination signals gracefully.

```
DECLARE
    CURSOR emp_cursor IS
        SELECT emp_id, salary FROM Employee;
    v_emp_id Employee.emp_id%TYPE;
    v_salary Employee.salary%TYPE;
    salary_zero EXCEPTION;
    no_data EXCEPTION;
BEGIN
    OPEN emp_cursor;
    FETCH emp_cursor INTO v_emp_id, v_salary;
    IF emp_cursor%NOTFOUND THEN
        RAISE no_data;
    END IF;
    WHILE emp_cursor%FOUND LOOP
        IF v_salary = 0 THEN
            RAISE salary_zero;
        END IF;
        DBMS_OUTPUT.PUT_LINE(
            'Processing Employee ID: ' || v_emp_id
        );
        FETCH emp_cursor INTO v_emp_id, v_salary;
    END LOOP;
    CLOSE emp_cursor;
EXCEPTION
    WHEN salary_zero THEN
```

```
        DBMS_OUTPUT.PUT_LINE('Error:  Salary is zero. Cannot process increment.');
        CLOSE emp_cursor;
    WHEN no_data THEN
        DBMS_OUTPUT.PUT_LINE('No  records found in Employee table.');
        CLOSE emp_cursor;
END;
```

```
Processing Employee ID: 1
Error: Salary is zero. Cannot process increment.
```

Outcomes:

- **Cursor Implementation:** Students will be able to design, implement, and manage cursors to solve row-wise processing problems.
- **Lifecycle Mastery:** Students will demonstrate the correct syntax for declaring, opening, fetching, and closing cursors.
- **Error Prevention:** Students will understand how to properly handle row-by-row processing exceptions and prevent memory leaks via deallocation.
- **Analytical Thinking:** Students will be able to apply cursor-based logic to solve real-world scenarios like multi-level payroll adjustments or data migrations.